STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID: LRC-26-0001	TEAM NAME: Bright Minds

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	8mm
Fluid density (ρ) =	1000kg/m^3
Dynamic Viscosity (μ) =	1x10^{-3}Pa.s
Required Flow range =	1-6 L/min
Nominal target flow for the live challenge =	3L/min

Lam RESEARCH

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity	1. Custom PCB
2. Pumps – 3 Quantity	2. ESP32 NodeMCU module
3. FT-01 Flow Transmitter – 01 Quantity	3. Pump Driver-1
4. Ball valve – 3 Quantity	4. Pump Driver-2
5. ON/OFF/XV Valve – 1 Quantity	5. Pump Driver-3
6. 10-mm OD tubing/blue – 9 meters/1 meter	6. 12 V DC power input (**SMPS**)
7. Microcontroller/PCB/ data-acquisition software	7. 12 VDC Solenoid-valve interface
8. Required electrical/ electronic accessories	8. Flow-sensor interface
	9. Float-switch interface
	10. ON/OFF-switch interface
	11. Pump-driver PWM/enable interfaces
	12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code**. The case-1 can be turned on using the available on/off Switch, to start the system.

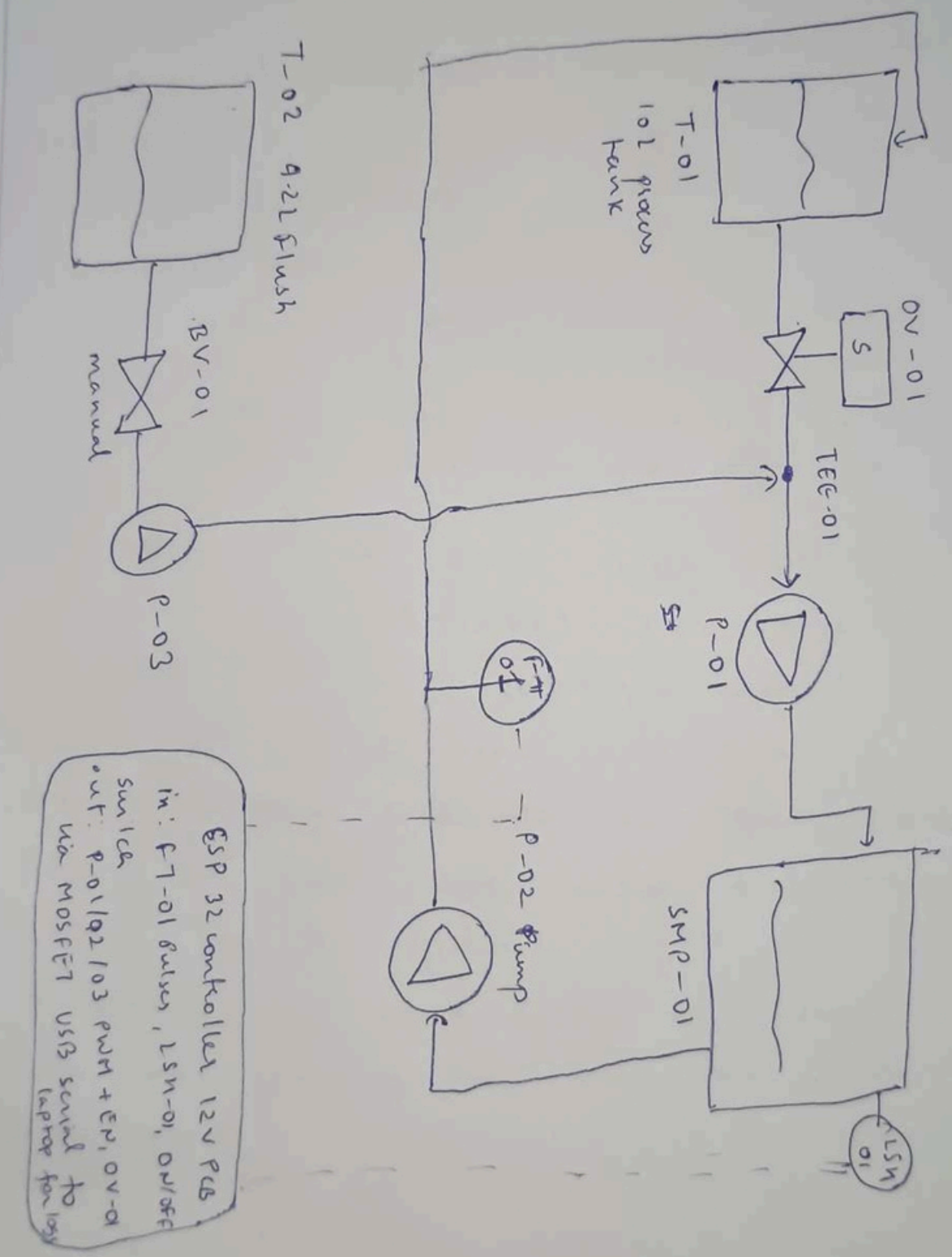
CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram


T-01
10L plain tank
OV-01
S
TEE-01
P-01
T-02 9.2L flush
BV-01
manual
P-03
F-T 01
P-02 Pump
SMP-01
LSH 01
ESP 32 controller 12V PCB
in: FT-01 pulses, LSH-01, ON/OFF switch
out: P-01/02/03 PWM + EN, OV-01 via MOSFET USB serial to laptop for log

Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

Flow is set using P-02 pump PMW duty and ball valve BV-02. Flow readings are taken from calibrated sensor.

Theoretical values :

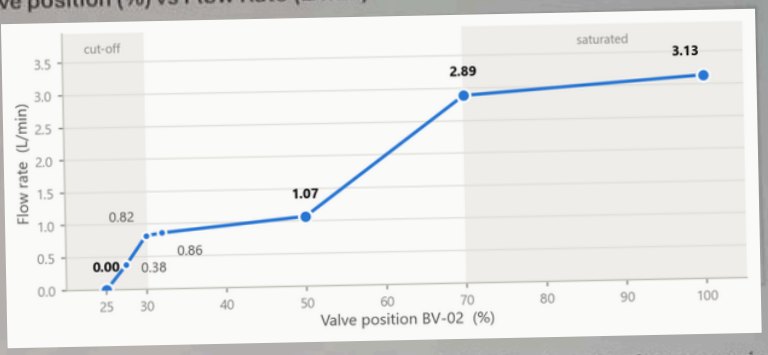
Q	v, 8mm	Re, 8mm	v, 5mm	Re, 5mm
1	0.322 m/s	2653	0.849 m/s	4244
3	0.995 m/s	7958	2.546 m/s	12732
6	1.989 m/s	15915	5.093 m/s	25465

To reach 1 L/min BV-02 is set at 50 % and P-02 at 100 %. Q = 1.07 L/min v = 0.909 m/s Deviation ~ 7.1 %. To reach around 3L/min BV-02 is set at 100 % and P-02 at 100 %. Q = 3.13 m/s v = 2.660 m/s Re = 13301 Dev 4.5 %. Since the flow rate maxed out around 3 L/min for the system, it was not possible to operate the same System at 6L/min so with max operating flow rate the deviation from pump flow rate is ~47.8 %. because the pump free flow rate is around 4.0 L/min so 6L/min is too optimistic flow rate nameplate for pump and it reduces significantly when FT-01 sensor is attached to it.

FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	0.00
50%	1.07
70%	2.89
100%	3.13



Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 =	0.9716
a =	0.012809 L/min per Hz
b =	-0.38382 L/min
Maximum calibration error =	6.86 %. (0.151 L/min, trial 4)
Average calibration error =	4.12 %. (0.096 L/min)

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	4500	30	150	0.75	1.500
2	4451	30	148.5	0.800	1.600
3	6426	30	214	1.200	2.400
4	6401	30	213.5	1.100	2.200
5	8160	30	272	1.500	3.000
6	8010	30	267	1.600	3.200

CALCULATIONS / OBSERVATIONS

Q2.3 - The return line was connected to 10 L containes where we marked 100 ml spacings. Trial 1-2 were run at 50 % pump duty, 3-4 at 75 % and 5-6 at 100 %, with BV-02 fully open throughout for a timed 30 s window while FT-01 were counted ove the same window, giving $f = N/t$ and $Q = V/t \times 60$. Volume is an independent measurement which is what makes the table usable as calibration data.

Q2.2 - $Q = a \cdot f + b$

Least squares on the above 6 pairs gives $a = 0.012809$ $b = -0.38382$, with $R^2 = 0.9716$. The implied k-factor is $60/a = 4684$ pulses per litre compared to 5880 value of datasheet. We founde out that highest percentage error is in trial 4 with $Q_{actual} = 2.2$ $Q_{fit} = 2.351$, hence percentage error 6.86 % while averge error was 4.12 %.

Q2.3 - Pump duty was held at 100 % and each point was sampled over 10 sec to get averge rate from $Q = af + b$.

General observation: (i) Below 27 % the valve is practically shut-off and flow is zero (at 27 % flow is 0.38 L/min) while above 70 % the curve saturate)

(ii) After removing the sensor and only testing the flow measure come out to be nearly 4 L/min making FT-01 a dominant restriction.

(iii) Below 27 %. the flow rate must to below 0.3 L/min which is below the rating of flow sensor FT-01 (YF-S401) hence we got 0.00 as our rating.

HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER

$$T_r = \frac{V}{Q} = \frac{0.5\,L}{2.5\,L/min} = 0.20\,min = 12\,s$$

(The formula above mentioned is incorrect dimensionally and according to theory).

1. Residence time represents average time a fluid element spends inside that section of tubing i.e. how long it takes liquid entering at inlet to leave at the outlet.
2. If Tubing volume is doubled while flow rate remains constant $T_r \propto V$ so residence time is also doubled.
3. Since $T_r \propto 1/Q$ so if flow rate is doubled residence time becomes halved
4. A change in upstream reaches the sensor only after about T_r seconds. We need to wait several residences time before concluding any faults or risks.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

Cond A is healthy operation. Condition B is valve restriction, can also be weak pump. For condition C it can be Pump failure, failed sensor, broken signal, blocked line or stuck valve. For this additional information like sensor check, voltage and signal level at PCB, motor current of the pump driver and the sound or vibration of the pump. valve state and what changed before that event. For Condition D it is expected flow as pump is turned off so no suction pressure. For condition E It can is due to unstable flow which can be due to air entrapment, cavitation, ~~to~~ unexpected changes in water level of tank, Low sump level or noise in the signal can also be jammed turbine of sensor.

HARDWARE LIST

Mechanical Hardware

Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow,** allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe. Mainly used for Pumps**
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line.**

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces**.
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller**. It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections**.
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board

ANNEXURE

Supporting graphs, telemetry and controller firmware

CONTENTS

A-1 Q2.1 Flow response to ball-valve position BV-02 ..... A-1

A-2 Q2.2 Experimental calibration of flow transmitter FT-01 ..... A-2

A-3 Q2.2 Calibration error by trial A-3

A-4 Live telemetry at the 3 L/min nominal target A-4

A-5 Telemetry during the autonomous Case 1 run A-5

A-6 Controller firmware - stage2_final.ino A-6

Graphs A-1 to A-3 are plotted from the serial log of the live run and from the six jug-and-stopwatch calibration trials; A-1 is the graph required by Q2.1, and A-2 and A-3 support the calibration equation and the error values reported in Q2.2. A-4 and A-5 are Serial Monitor captures of that telemetry as it was recorded. A-6 is the complete controller firmware as submitted.

Figure A-1 Q2.1 Flow response to ball-valve position BV-02

P-02 held at 100 % PWM duty. Each point is a settled averaging window taken from the serial log; flow computed from the calibrated sensor equation Q = 0.012809 f - 0.38382. The four filled markers are the valve positions required by Q2.1.

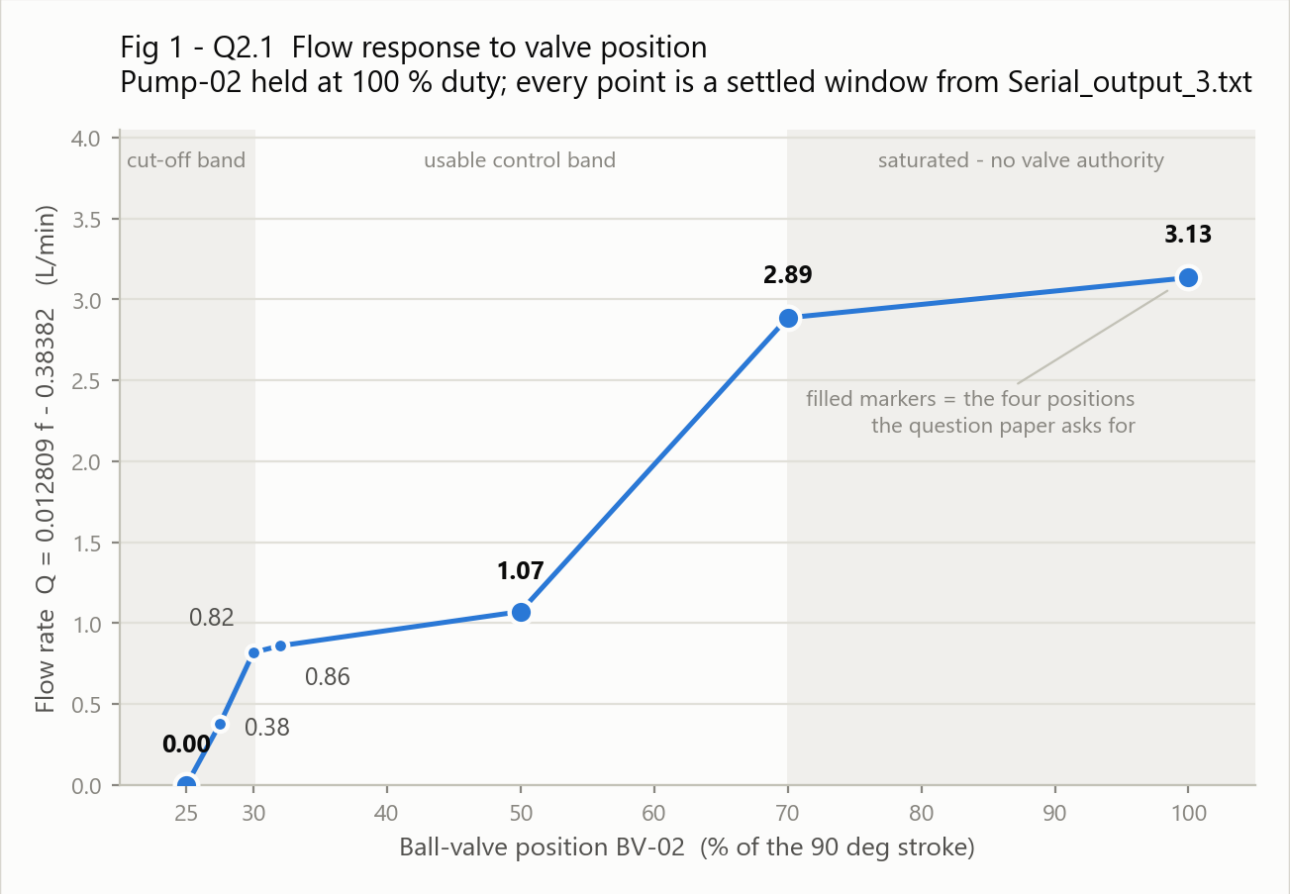


Figure A-2 Q2.2 Experimental calibration of flow transmitter FT-01

Least-squares fit of six jug-and-stopwatch trials. Actual flow Q = V / t x 60 is plotted against the measured FT-01 pulse frequency, giving a = 0.012809 L/min per Hz, b = -0.38382 L/min, R2 = 0.9716 and K = 4684 pulses per litre.

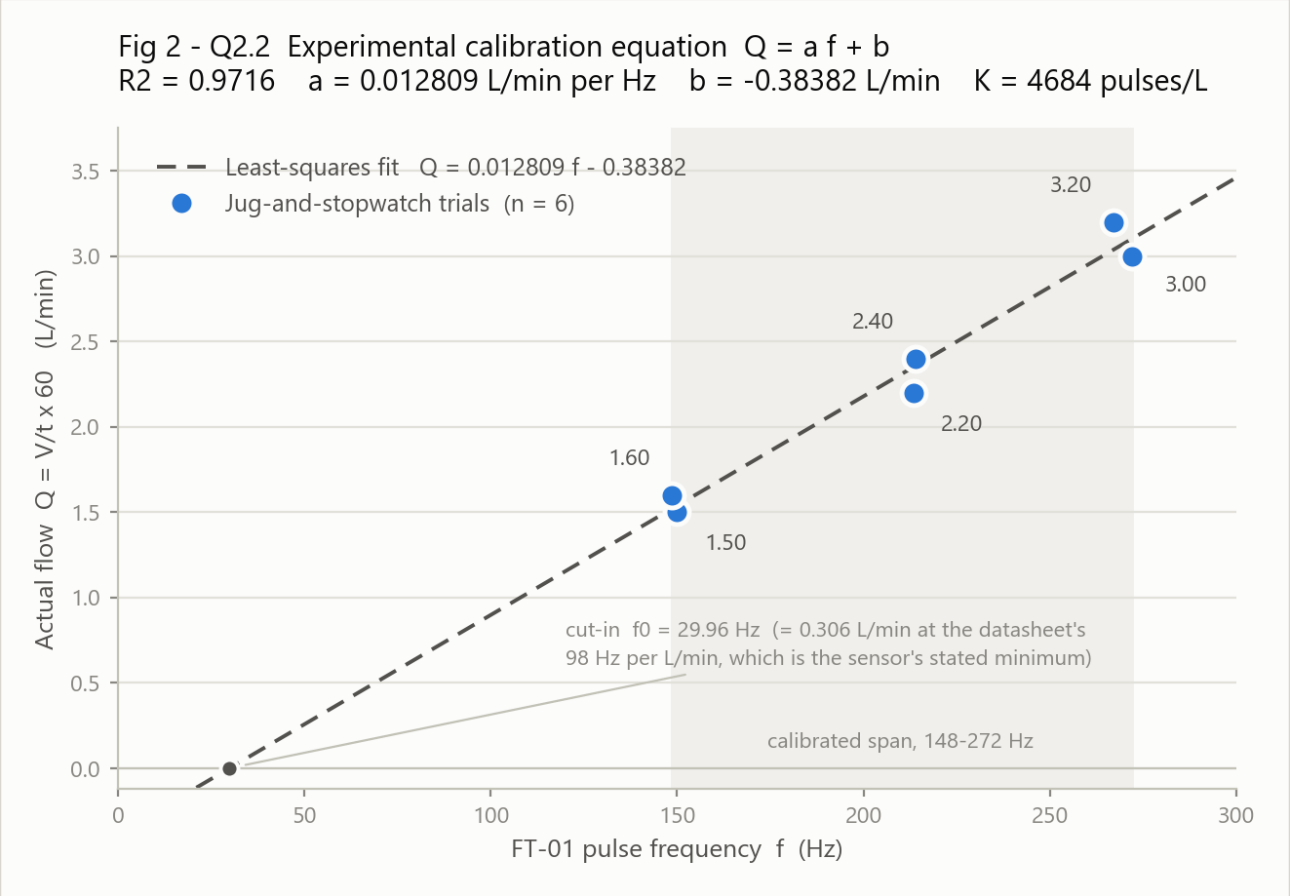


Figure A-3 Q2.2 Calibration error by trial

Error of the fitted equation against each measured trial, |Q_fit - Q_actual| / Q_actual. Maximum calibration error 6.86 % (0.151 L/min, trial 4); average calibration error 4.12 % (0.096 L/min).

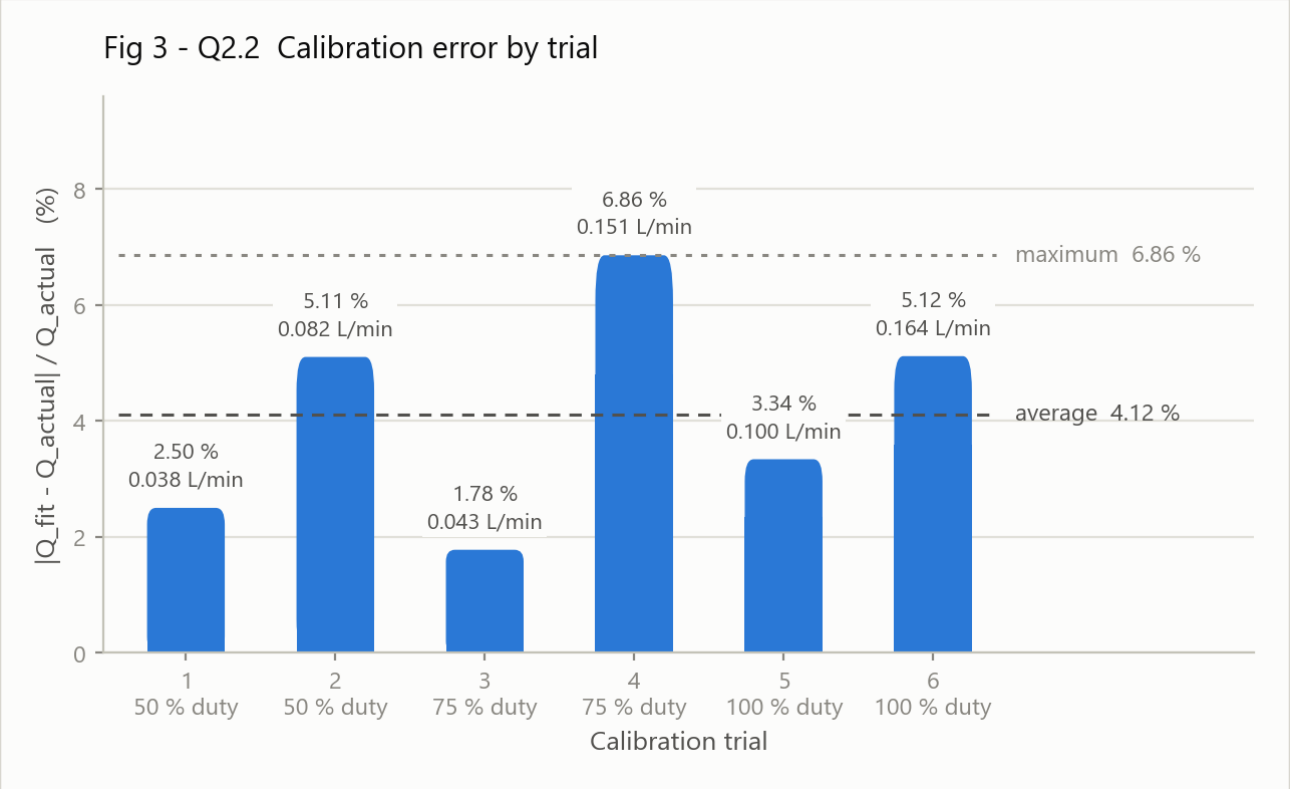


Figure A-4 Live telemetry at the 3 L/min nominal target

Serial Monitor capture taken while the system held the nominal target flow. Each row is one 500 ms measurement window: elapsed time, cumulative FT-01 pulse count, pulse frequency, flow from the calibrated equation in L/min and in mL/s, the running totaliser in litres, and the float-switch state. Flow holds between 3.109 and 3.162 L/min and the totaliser reaches 3.1004 L at 59.5 s.

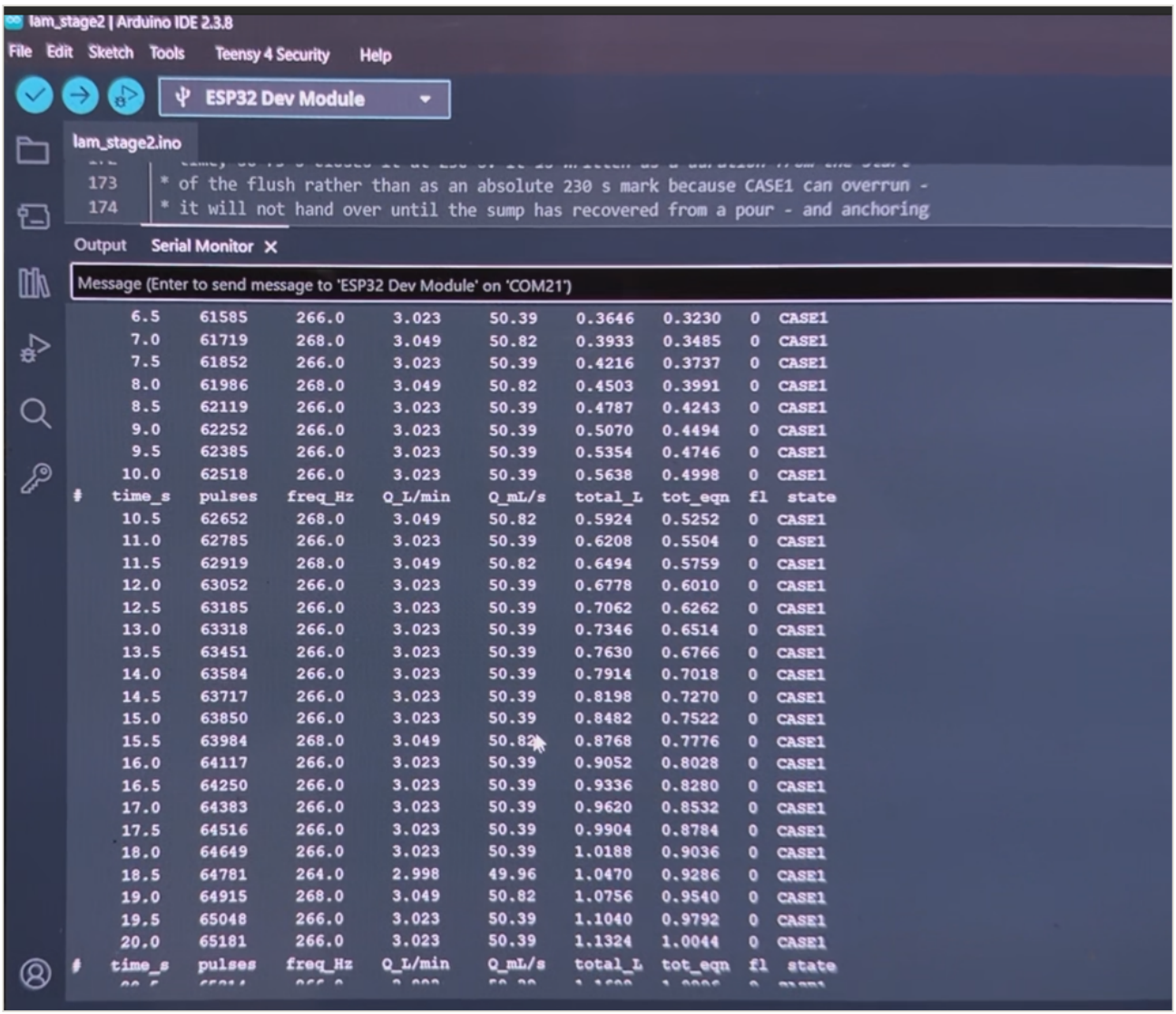
Output Serial Monitor ×

Message (Enter to send message to 'ESP32 Dev Module' on 'COM21')

	46.5	12240	264.0	3.135	52.26	2.4220	0
	47.0	12373	266.0	3.162	52.70	2.4483	0
	47.5	12504	262.0	3.109	51.81	2.4742	0
	48.0	12636	264.0	3.135	52.26	2.5004	0
	48.5	12768	264.0	3.135	52.26	2.5265	0
	49.0	12899	262.0	3.109	51.81	2.5524	0
	49.5	13031	264.0	3.135	52.26	2.5785	0
	50.0	13163	264.0	3.135	52.26	2.6047	0
#	time_s	pulses	freq_Hz	Q_L/min	Q_mL/s	total_L	float
	50.5	13295	264.0	3.135	52.26	2.6308	0
	51.0	13426	262.0	3.109	51.81	2.6567	0
	51.5	13558	264.0	3.135	52.26	2.6828	0
	52.0	13690	264.0	3.135	52.26	2.7090	0
	52.5	13823	266.0	3.162	52.70	2.7353	0
	53.0	13955	264.0	3.135	52.26	2.7614	0
	53.5	14086	262.0	3.109	51.81	2.7873	0
	54.0	14218	264.0	3.135	52.26	2.8135	0
	54.5	14350	264.0	3.135	52.26	2.8396	0
	55.0	14481	262.0	3.109	51.81	2.8655	0
	55.5	14614	266.0	3.162	52.70	2.8919	0
	56.0	14746	264.0	3.135	52.26	2.9180	0
	56.5	14878	264.0	3.135	52.26	2.9441	0
	57.0	15009	262.0	3.109	51.81	2.9700	0
	57.5	15141	264.0	3.135	52.26	2.9961	0
	58.0	15272	262.0	3.109	51.81	3.0221	0
	58.5	15404	264.0	3.135	52.26	3.0482	0
	59.0	15537	266.0	3.162	52.70	3.0745	0
	59.5	15668	262.0	3.109	51.81	3.1004	0

Figure A-5 Telemetry during the autonomous Case 1 run

The same telemetry logged by the controller while the state machine was running Case 1 unattended, with the state column reading CASE1 throughout. Flow is steady at 3.023 - 3.049 L/min, the float column stays at 0 so no level cut-off occurred in this window, and both totalisers advance together: pulse count / K against the integrated calibration equation.



Annexure A-6 Controller firmware - stage2_final.ino

Complete source for the ESP32 controller as submitted. Single autonomous build: a short button press runs Case 1 and then Case 2 to completion, and a 1.5 s hold aborts to a safe state. Serial carries telemetry only and accepts no commands, so the sequence cannot be driven from the laptop.

```
/* ---------------------------------------------------------------------------
 * LAM Research Challenge 3.0 - Stage 2
 * ESP32 NodeMCU-32S on the LAM Stage 2 PCB (REV 1.0)
 *
 * Case 1  Process fluid transfer with automatic sump level control.
 * Case 2  Flush with all three pumps, then shutdown when the flush tank empties.
 *
 * Push button: short press starts the sequence, 1.5 s hold aborts it.
 * Serial @ 115200 is telemetry only - the system takes no commands.
 * ------------------------------------------------------------------------- */

#include <Arduino.h>

/* ------------------------------------------------------------------ pin map */
const int PIN_P1_RPWM = 33, PIN_P1_LPWM = 32, PIN_P1_REN = 26, PIN_P1_LEN = 13;
const int PIN_P2_RPWM = 19, PIN_P2_LPWM = 18, PIN_P2_REN = 17, PIN_P2_LEN = 16;
const int PIN_P3_RPWM = 25, PIN_P3_LPWM = 23, PIN_P3_REN = 22, PIN_P3_LEN = 21;

const int PIN_OV    = 27;   /* OV-01 solenoid */
const int PIN_FLOW  = 14;   /* FT-01 pulse input */
const int PIN_FLOAT = 35;   /* input-only, PCB pull-up */
const int PIN_SW    = 34;   /* input-only, PCB pull-up */
const int PIN_LED   =  2;

/* Both contacts pull to GND, so closed reads LOW. */
const bool FLOAT_HIGH_LEVEL_STATE = LOW;
const bool SWITCH_PRESSED_STATE   = LOW;
const bool OV_ON_STATE            = HIGH;

/* Driver channels in process-role order. Pump-02's R_PWM trace is absent on
 * this PCB, so it is modulated on L_PWM and its motor leads are swapped. */
const int CH_FLUSH = 0;   /* U2 -> Pump-03 */
const int CH_DRAIN = 1;   /* U3 -> Pump-02 */
const int CH_FILL  = 2;   /* U4 -> Pump-01 */

/* ---------------------------------------------------------------- calibration */
/* Q = CAL_A * f + CAL_B, six-point least-squares fit. R2 = 0.9716. */
const float CAL_A =  0.012809f;
const float CAL_B = -0.38382f;
const float K_PULSES_PER_L = 4684.0f;

/* ------------------------------------------------------------------ settings */
const uint8_t DUTY_FILL  =  50;
const uint8_t DUTY_DRAIN = 100;
const uint8_t DUTY_FLUSH =  50;

const uint32_t PWM_HZ   = 10000;
const uint8_t  PWM_BITS = 10;

const uint32_t WINDOW_MS       = 500;
const uint32_t HEADER_EVERY_MS = 10000;

const uint32_t SW_DEBOUNCE_MS      = 50;
const uint32_t LONG_PRESS_MS       = 1500;
const uint32_t FLOAT_DEBOUNCE_MS   = 300;
const uint32_t FLOAT_CLEAR_HOLD_MS = 3000;

const uint32_t CASE1_DURATION_S = 120;   /* transfer + level-control demo */
const uint32_t DRAIN_S          = 15;    /* clear the sump back to T-01 */
const uint32_t WAIT_BV_S        = 20;    /* operator opens BV-01 */

/* The flush tank has no level sensor, so "empty" is inferred from flow
 * collapsing - but only after enough volume has passed that the 4.2 L tank
 * could actually be empty, which rules out a false stop on the sump heel. */
const uint32_t FLUSH_GRACE_S       = 12;
const float    FLUSH_END_Q_LMIN    = 0.40f;
const uint32_t FLUSH_END_CONFIRM_S = 4;
const float    FLUSH_END_MIN_VOL_L = 2.5f;
const uint32_t FLUSH_MAX_S         = 150;  /* backstop if flow never collapses */

```

```
const uint32_t SEQUENCE_MAX_S = 1200;      /* global watchdog */

/* Staggered starts so three motors do not draw inrush together. */
const uint32_t FLUSH_STAGGER_1_MS = 400;
const uint32_t FLUSH_STAGGER_2_MS = 800;

enum State { ST_IDLE, ST_CASE1, ST_DRAIN, ST_WAIT_BV, ST_FLUSH, ST_DONE };

/* ---------------------------------------------------------------- actuators */
struct Pump {
  const char* name;
  int pwmPin;        /* the input that is modulated */
  int lowPin;        /* the opposite input, held LOW */
  int ren, len, ch;
  bool     running;
  uint8_t  duty;
};

Pump pumps[3] = {
  { "Pump-03", PIN_P1_RPWM, PIN_P1_LPWM, PIN_P1_REN, PIN_P1_LEN, 0, false, DUTY_FLUSH },
  { "Pump-02", PIN_P2_LPWM, PIN_P2_RPWM, PIN_P2_REN, PIN_P2_LEN, 1, false, DUTY_DRAIN },
  { "Pump-01", PIN_P3_RPWM, PIN_P3_LPWM, PIN_P3_REN, PIN_P3_LEN, 2, false, DUTY_FILL  }
};

/* Which pumps the float may cut. Pump-02 is the only sump outlet and is never
 * cut, or the level could never fall again. */
bool protect[3] = { false, false, false };

bool ovOpen      = false;
bool levelAllows = true;

/* ------------------------------------------------------------------ inputs */
bool     floatHigh = false, floatCand = false;
uint32_t floatChangeT = 0, floatClearSince = 0;

bool     swPressed = false, swCand = false;
uint32_t swChangeT = 0, pressStart = 0;
bool     longFired = false;
bool     evShortPress = false, evLongPress = false;

volatile uint32_t gPulses     = 0;
volatile uint32_t gLastEdgeUs = 0;

uint32_t lastWinMs = 0, lastWinPulses = 0, lastHeaderMs = 0;
uint32_t flowStartMs = 0, totBasePulses = 0;
double   volEqnL = 0;   /* litres by integrating Q = a*f + b */
float    lastQ = 0;

State    state = ST_IDLE;
uint32_t stateSince = 0, seqStart = 0, flushLowSince = 0;
bool     flushEmptyConfirmed = false;
double   flushBaseL = 0;
uint8_t  flushStaggerStep = 0;

void IRAM_ATTR onFlowPulse() {
  uint32_t now = micros();
  if (now - gLastEdgeUs < 300) return;   /* ~3.3 kHz ceiling; 6 L/min is ~590 Hz */
  gLastEdgeUs = now;
  gPulses++;
}

double volPulseL() { return (double)(gPulses - totBasePulses) / K_PULSES_PER_L; }

const char* stateName() {
  switch (state) {
    case ST_IDLE:    return "IDLE";
    case ST_CASE1:   return "CASE1";
    case ST_DRAIN:   return "DRAIN";
    case ST_WAIT_BV: return "WAIT_BV";
    case ST_FLUSH:   return "FLUSH";
    default:         return "DONE";
  }
}

/* ------------------------------------------------------------------ output */
bool pumpShouldRun(int i) {
  if (!pumps[i].running) return false;
  if (protect[i] && !levelAllows) return false;
  return true;
```

```
}

void applyPump(int i) {
  Pump &p = pumps[i];
  uint8_t  pct  = pumpShouldRun(i) ? p.duty : 0;
  uint32_t maxv = (1u << PWM_BITS) - 1;
  uint32_t duty = (maxv * (uint32_t)pct) / 100;

#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(p.pwmPin, duty);
#else
  ledcWrite(p.ch, duty);
#endif

  digitalWrite(p.lowPin, LOW);
  digitalWrite(p.ren, pct > 0 ? HIGH : LOW);
  digitalWrite(p.len, pct > 0 ? HIGH : LOW);
}

void applyAll() { for (int i = 0; i < 3; i++) applyPump(i); }

void setPump(int i, bool on) {
  pumps[i].running = on;
  applyPump(i);
  Serial.printf("# %s %s (duty %u %%)\n", pumps[i].name, on ? "START" : "STOP", pumps[i].duty);
}

void setValve(bool open) {
  ovOpen = open;
  digitalWrite(PIN_OV, open ? OV_ON_STATE : !OV_ON_STATE);
  Serial.printf("# OV-01 %s\n", open ? "OPEN" : "CLOSED");
}

void stopAll() {
  for (int i = 0; i < 3; i++) { pumps[i].running = false; applyPump(i); }
  ovOpen = false;
  digitalWrite(PIN_OV, !OV_ON_STATE);
  Serial.println("# ALL STOPPED - pumps off, OV-01 closed");
}

/* ------------------------------------------------------------------ sensing */
void updateSwitch(uint32_t now) {
  bool raw = (digitalRead(PIN_SW) == SWITCH_PRESSED_STATE);
  if (raw != swCand) { swCand = raw; swChangeT = now; }

  if (now - swChangeT >= SW_DEBOUNCE_MS && swPressed != swCand) {
    swPressed = swCand;
    if (swPressed) { pressStart = now; longFired = false; }
    else if (!longFired) evShortPress = true;
  }

  if (swPressed && !longFired && now - pressStart >= LONG_PRESS_MS) {
    longFired   = true;
    evLongPress = true;
  }
}

void updateFloat(uint32_t now) {
  bool raw = (digitalRead(PIN_FLOAT) == FLOAT_HIGH_LEVEL_STATE);
  if (raw != floatCand) { floatCand = raw; floatChangeT = now; }

  if (now - floatChangeT >= FLOAT_DEBOUNCE_MS && floatHigh != floatCand) {
    floatHigh = floatCand;
    Serial.printf("# FLOAT -> %s\n", floatHigh ? "HIGH (sump full)" : "clear");
  }

  bool before = levelAllows;
  if (floatHigh) { levelAllows = false; floatClearSince = 0; }
  else if (!levelAllows) {
    if (floatClearSince == 0) floatClearSince = now;
    if (now - floatClearSince >= FLOAT_CLEAR_HOLD_MS) levelAllows = true;
  }

  if (before != levelAllows) {
    Serial.printf("# LEVEL PROTECTION: protected pumps %s\n",
                  levelAllows ? "released" : "CUT OFF");
    applyAll();
  }
}
```

```
229
230 void updateFlow(uint32_t now) {
231   uint32_t dt = now - lastWinMs;
232   if (dt < WINDOW_MS) return;
233
234   uint32_t p  = gPulses;
235   uint32_t dN = p - lastWinPulses;
236   lastWinPulses = p; lastWinMs = now;
237
238   float freqHz = dN * 1000.0f / dt;
239   float Qlmin  = (dN == 0) ? 0.0f : CAL_A * freqHz + CAL_B;
240   if (Qlmin < 0) Qlmin = 0;
241   lastQ = Qlmin;
242   volEqnL += Qlmin * dt / 60000.0;
243
244   if (now - lastHeaderMs >= HEADER_EVERY_MS) {
245     lastHeaderMs = now;
246     Serial.println("#   time_s   pulses   freq_Hz   Q_L/min    Q_mL/s   total_L  tot_eqn  fl  state");
247   }
248
249   Serial.printf("  %7.1f  %7lu  %8.1f  %8.3f  %8.4f   %d  %s\n",
250                 (now - flowStartMs) / 1000.0, (unsigned long)p, freqHz,
251                 Qlmin, volPulseL(), floatHigh ? 1 : 0, stateName());
252 }
253
254 /* ------------------------------------------------------------ state machine */
255 void enter(State s, uint32_t now) {
256   state = s; stateSince = now;
257   Serial.printf("\n# ======== STATE -> %s ========\n", stateName());
258 }
259
260 void finishRun(uint32_t now, const char* reason) {
261   stopAll();
262   Serial.printf("\n# ####### RUN COMPLETE - %s #######\n", reason);
263   Serial.printf("# run time     : %.1f s\n", (now - seqStart) / 1000.0);
264   Serial.printf("# flush volume : %.4f L\n", volPulseL() - flushBaseL);
265   Serial.printf("# total volume : %.4f L\n", volPulseL());
266   Serial.println("# press the button once to reset.");
267 }
268
269 void sequenceTick(uint32_t now) {
270   bool shortPress = evShortPress; evShortPress = false;
271   bool longPress  = evLongPress;  evLongPress  = false;
272
273   if (longPress && state != ST_IDLE) {
274     Serial.println("\n# !! BUTTON HELD - ABORTING TO SAFE STATE !!");
275     stopAll();
276     enter(ST_IDLE, now);
277     return;
278   }
279
280   if (state != ST_IDLE && state != ST_DONE && now - seqStart > SEQUENCE_MAX_S * 1000UL) {
281     Serial.println("\n# !! SEQUENCE WATCHDOG EXPIRED !!");
282     stopAll();
283     enter(ST_DONE, now);
284     return;
285   }
286
287   switch (state) {
288
289     case ST_IDLE:
290       if (shortPress) {
291         seqStart      = now;
292         totBasePulses = gPulses;
293         volEqnL       = 0;
294         lastWinMs     = now;
295         lastWinPulses = gPulses;
296         flowStartMs   = now;
297         lastHeaderMs  = now - HEADER_EVERY_MS;
298         flushBaseL          = 0;
299         flushLowSince       = 0;
300         flushEmptyConfirmed = false;
301
302         Serial.println("\n# ########## CASE 1: PROCESS FLUID TRANSFER ##########");
303         protect[CH_FILL]  = true;
304         protect[CH_DRAIN] = false;
305         protect[CH_FLUSH] = false;
306         setValve(true);
307         setPump(CH_DRAIN, true);
```

```
        setPump(CH_FILL,  true);
        enter(ST_CASE1, now);
      }
      break;

    case ST_CASE1:
      /* Hand over only once the dwell is done and the sump has recovered, so
       * the transition never lands in the middle of a level cut-off. */
      if (now - stateSince >= CASE1_DURATION_S * 1000UL && levelAllows && !floatHigh) {
        Serial.println("\n# ######## DRAIN: isolating the process tank ########");
        setValve(false);
        setPump(CH_FILL, false);   /* its suction is dead-ended with OV-01 shut */
        enter(ST_DRAIN, now);
      }
      break;

    case ST_DRAIN:
      /* Pump-02 pushes the process fluid left in the sump back to T-01 before
       * any flush water arrives, so the two mix as little as possible. */
      if (now - stateSince >= DRAIN_S * 1000UL) {
        setPump(CH_DRAIN, false);
        Serial.println("#");
        Serial.println("#   >>> OPEN THE MANUAL BALL VALVE BV-01 NOW <<<");
        Serial.printf ("#   >>> the flush starts in %lu s\n", (unsigned long)WAIT_BV_S);
        Serial.println("#");
        enter(ST_WAIT_BV, now);
      }
      break;

    case ST_WAIT_BV:
      if (now - stateSince >= WAIT_BV_S * 1000UL) {
        Serial.println("\n# ########## CASE 2: FLUSH ##########");
        protect[CH_FILL]  = true;
        protect[CH_FLUSH] = true;   /* it feeds Pump-01's suction through the tee */
        protect[CH_DRAIN] = false;
        flushBaseL       = volPulseL();
        flushLowSince    = 0;
        flushStaggerStep = 0;
        enter(ST_FLUSH, now);
      }
      break;

    case ST_FLUSH:
      if (flushStaggerStep == 0) {
        setPump(CH_DRAIN, true);
        flushStaggerStep = 1;
      } else if (flushStaggerStep == 1 && now - stateSince >= FLUSH_STAGGER_1_MS) {
        setPump(CH_FILL, true);
        flushStaggerStep = 2;
      } else if (flushStaggerStep == 2 && now - stateSince >= FLUSH_STAGGER_2_MS) {
        setPump(CH_FLUSH, true);
        flushStaggerStep = 3;
        Serial.println("# all three pumps running simultaneously");
      }

      {
        float flushedL = volPulseL() - flushBaseL;
        bool  armed    = (now - stateSince >= FLUSH_GRACE_S * 1000UL) &&
                         (flushedL >= FLUSH_END_MIN_VOL_L);

        if (lastQ < FLUSH_END_Q_LMIN) {
          if (flushLowSince == 0) flushLowSince = now;
          if (armed && now - flushLowSince >= FLUSH_END_CONFIRM_S * 1000UL) {
            flushEmptyConfirmed = true;
            Serial.printf("# FLUSH TANK EMPTY at %.1f s, %.3f L flushed\n",
                          (now - seqStart) / 1000.0, flushedL);
            finishRun(now, "FLUSH WATER EXHAUSTED");
            enter(ST_DONE, now);
          }
        } else {
          flushLowSince = 0;
        }
      }

      if (!flushEmptyConfirmed && now - stateSince >= FLUSH_MAX_S * 1000UL) {
        finishRun(now, "FLUSH TIME LIMIT - tank not confirmed empty");
        enter(ST_DONE, now);
      }
      break;
```

```
387
388     case ST_DONE:
389       if (shortPress) {
390         Serial.println("# reset - ready to run again");
391         enter(ST_IDLE, now);
392       }
393       break;
394   }
395 }
396
397 /* ------------------------------------------------------------ setup / loop */
398 void setup() {
399   Serial.begin(115200);
400   delay(300);
401
402   pinMode(PIN_LED,   OUTPUT);
403   pinMode(PIN_OV,    OUTPUT);
404   pinMode(PIN_FLOW,  INPUT_PULLUP);
405   pinMode(PIN_FLOAT, INPUT);   /* input-only pins: the PCB supplies the pull-up */
406   pinMode(PIN_SW,    INPUT);
407
408   for (int i = 0; i < 3; i++) {
409     pinMode(pumps[i].lowPin, OUTPUT); digitalWrite(pumps[i].lowPin, LOW);
410     pinMode(pumps[i].ren,    OUTPUT); digitalWrite(pumps[i].ren,    LOW);
411     pinMode(pumps[i].len,    OUTPUT); digitalWrite(pumps[i].len,    LOW);
412 #if ESP_ARDUINO_VERSION_MAJOR >= 3
413     ledcAttach(pumps[i].pwmPin, PWM_HZ, PWM_BITS);
414 #else
415     ledcSetup(pumps[i].ch, PWM_HZ, PWM_BITS);
416     ledcAttachPin(pumps[i].pwmPin, pumps[i].ch);
417 #endif
418   }
419
420   attachInterrupt(digitalPinToInterrupt(PIN_FLOW), onFlowPulse, RISING);
421
422   /* Seed the inputs so the first read is not reported as a change. */
423   floatCand = floatHigh = (digitalRead(PIN_FLOAT) == FLOAT_HIGH_LEVEL_STATE);
424   swCand    = swPressed = (digitalRead(PIN_SW)    == SWITCH_PRESSED_STATE);
425   levelAllows = !floatHigh;
426   lastWinMs   = flowStartMs = millis();
427
428   stopAll();
429
430   Serial.println("\n# ====================================================");
431   Serial.println("# LAM Research Challenge 3.0 - Stage 2");
432   Serial.printf ("# CAL: Q = %.6f*f %+.5f   K = %.0f pulses/L\n",
433                  CAL_A, CAL_B, K_PULSES_PER_L);
434   Serial.println("# Press the button to start. Hold 1.5 s to abort.");
435   Serial.println("# ====================================================");
436 }
437
438 void loop() {
439   uint32_t now = millis();
440
441   updateSwitch(now);
442   updateFloat(now);
443   updateFlow(now);
444   sequenceTick(now);
445
446   /* steady = running, fast blink = open BV-01, slow blink = idle or done */
447   bool anyRunning = pumpShouldRun(0) || pumpShouldRun(1) || pumpShouldRun(2);
448   if      (state == ST_WAIT_BV) digitalWrite(PIN_LED, (now / 120) % 2);
449   else if (anyRunning)          digitalWrite(PIN_LED, HIGH);
450   else                          digitalWrite(PIN_LED, (now / 1000) % 2);
451 }
```